MOJO · MAX · VULKAN · ADRENO 710

Mojo and MAX on a phone GPU

How we ran Mojo kernels and Llama 3.2 1B on a Qualcomm Adreno 710: first by retargeting Mojo's output, then through a native compiler target, and how far MAX itself is from the phone.

moto g stylus 5G (2024) · Adreno 710 · Mojo 1.2 and a 1.3 build from source · MAX 26.7 · October 2026

THE QUESTION

Mojo and MAX ship for NVIDIA, AMD and Apple GPUs. Can they reach a phone GPU that only speaks Vulkan and OpenCL?

The device

moto g stylus 5G (2024) with an Adreno 710 GPU, driven over adb from a Mac.

Three drivers

Qualcomm’s stock Vulkan 1.1, Mesa Turnip (Vulkan 1.4, side-loaded) and Qualcomm OpenCL 3.0.

The toolchain

Mojo 1.2 and MAX 26.7 as shipped; later, the Mojo compiler built from source.

Five acts

ACT 1

Retarget

Rewrite Mojo’s LLVM IR into Vulkan and OpenCL. 8 kernels pass on 3 drivers.

ACT 2

Llama

Llama 3.2 1B decode: 48 of 48 tokens match PyTorch, 283 ms a token.

ACT 3

MAX

ResNet-18 through MAX on the Mac. No road from MAX to the phone.

ACT 4

Native target

Mojo built from source with an Adreno target. Same results, no retargeter.

ACT 5

MAX runtime

The compiler side ports. The runtime and the model compiler do not, yet.

> Orange: our own tools around Mojo. Blue: Mojo’s compiler itself, extended.

Act 1 · Borrow Mojo’s LLVM IR



OpenCL branch

mojo2cl.py writes OpenCL C from the same IR, because Qualcomm’s OpenCL accepts no SPIR-V.

On the phone

A small C runner loads the shader, binds the buffers, dispatches and checks every output element.

Why AMD’s gfx90a: its warps are 64 lanes wide, like Adreno’s subgroups, so WARP_SIZE comes out right.

What the retargeter rewrites

In Mojo’s AMD kernel IR	In the Vulkan shader
Raw pointer arguments	Storage buffers at (set 0, binding k)
Scalar arguments	64-bit slots of a params buffer
AMD thread and block registers	SPIR-V builtins: LocalInvocationId, WorkgroupId
Pointer arithmetic	(buffer, element index) pairs, rebuilt at each access
Shared arrays	Workgroup variables
DPP and bpermute lane moves	Subgroup shuffles; warp.sum becomes one reduction
s_barrier	OpControlBarrier, semantics patched in the binary
64-bit integers	32-bit for the stock driver, which has no int64

The core problem: Vulkan has logical addressing. A shader may not do pointer arithmetic.

Bugs in the way, and how we got around them

LLVM 22

Barrier semantics Vulkan rejects

Patched in the SPIR-V binary.

LLVM 22

Loop strength reduction crashes it

llc -disable-lsr.

LLVM 22

if/elif chain, invalid structured CFG

Kernel rewritten as one if/else.

Stock driver

Two 16-bit shared arrays overlap

Pad to 4 bytes per element.

Stock driver

No reconvergence after loops

Uniform loops, shared-memory reductions.

Stock driver

No 64-bit integers

Index math narrowed to 32 bits.

> Each one stopped a kernel cold or gave wrong answers. None is Mojo’s fault.

Eight kernels pass on all three drivers

SGEMM, 1024 × 1024 × 1024 fp32, ms per run

Kernel	Turnip	Stock	OpenCL
Naive	669	200	409
Shared tiles	233	293	396
fp16 tiles	213	293	343
Register tiles	**32.5**	65.7	67.0

Also passing: vector add, vec4 loads, warp sum, block sum.

66

GFLOP/s, the best SGEMM: register tiles on Mesa Turnip

Act 2 · Llama 3.2 1B on the phone’s GPU

Export
fp16, fused q/k/v and gate/up: 2.47 GB, 103 tensors

7 Mojo kernels
embed, rmsnorm, gemv, gemv_add, rope_kv, attention, silu_mul

llama_vk.c
398 lines of C: buffers, pipelines, 150 dispatches a token

Check
tokens and logits against PyTorch fp32

Small kernels

All of decode is 219 lines of Mojo. The GEMVs read nearly every weight byte, and end in warp.sum.

Driver limits shape the export

The stock driver binds at most 128 MB, so the 525 MB embedding table is split into 4 chunks.

LLAMA 3.2 1B, DECODED ON THE ADRENO 710

48/48

generated tokens identical to PyTorch

283 ms

per token, 3.5 tokens a second, on both Vulkan drivers

55%

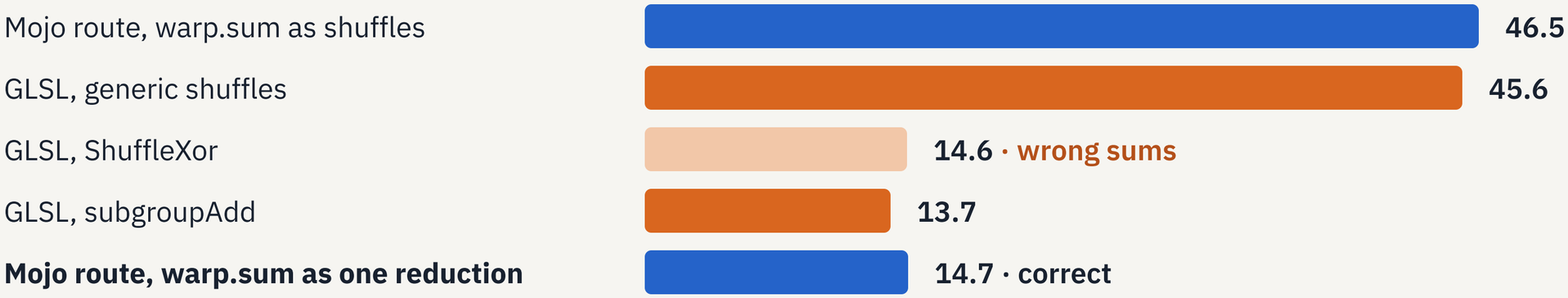
of the memory-bandwidth limit, 156 ms a token

> “The sky appears blue because of a phenomenon called Rayleigh scattering, in which shorter wavelengths of light, such as blue and violet, are scattered more…”

Mojo and MAX on Adreno · 9 · prompt of 46 tokens in 11.5 s · logits within 6.2e-5 of PyTorch

A 4.9× slowdown was one lowering

Stock driver · one 64 MB fp16 GEMV, ms per dispatch · the same kernel in GLSL as the yardstick



> Llama on the stock driver: **1,392 → 285 ms a token**, still 48 of 48 tokens right.

Act 3 · How MAX runs a model



ResNet-18 through MAX 26.7	Result
Mac CPU	**18 ms** an image · error 8.9e-7 against NumPy · same top-1 class
Compile time	66.5 s the first time, 1.0 s from the cache
Mac GPU (Apple)	compile failed, even for one relu: Xcode’s Metal compiler is not installed
Adreno 710	no target · and no ONNX import in 26.7 either

Where MAX stopped in 26.7

No SPIR-V target

Mojo’s compiler linked LLVM’s NVPTX, AMDGPU and Apple backends, not SPIR-V.

No Android runtime

MAX shipped for macOS and Linux hosts, with no Vulkan or OpenCL device.

Opaque internals

The 1.2 binary cannot dump Mojo’s MLIR, so a retargeter had to start at LLVM IR.

Vendor-tuned kernels

MAX’s library kernels lean on tensor cores, matrix cores and async copies.

> So Llama ran on our own C runtime, and every kernel went through the retargeter.

ACT 4 · THE COMPILER OPENED UP

18 Aug 2026

Modular open-sourced the Mojo compiler, a week after Mojo 1.0. The graph compiler, runtime and kernels sit beside it in modular/modular.

Codegen plugin
emits code for a new device

stdlib hooks
thread ids, math, printing

MAX hooks
warp size, barriers, shuffles

Driver plugin
runs code on the device

Mojo and MAX on Adreno · 13 · Apache 2.0 with LLVM exceptions · there is no Mojo 2.0; the latest release is 1.2

What we added: an adreno710 target

C++ · 2,129 LINES

In the compiler

Target traits, kernel marking, and a backend whose 1,660-line pass turns each kernel into a Vulkan entry point.

MOJO · 406 LINES

Two plugins

stdlib: ids as SPIR-V builtins, math as GLSL.std.450. MAX: 64-lane warps, barrier, shuffles.

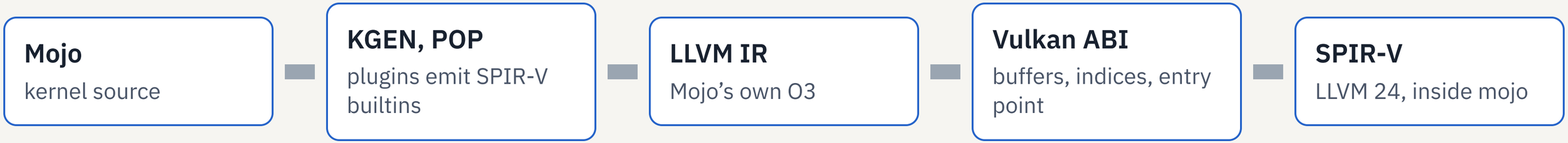
MAX · 33 LINES

One new hook

warp_sum_fn: a target can give warp.sum one native reduction, so no pattern matching.

```
$ mojo build --emit asm --target-accelerator adreno710 kernel.mojo
# one ready-to-load .spv per kernel, validated by spirv-val
```

The native pipeline, inside one compiler



What the target gains

The kernel is compiled for the Adreno from the start: its warp size, builtins, math and barrier are chosen by the target, with full types.

What disappears

The AMD stand-in target, text IR parsing, lane-move pattern matching, the barrier binary patch.

What it took to match the retargeter

Problem	Effect	Fix
Index as a 64-bit byte offset, then divided	naive SGEMM 2.6× slower	index from the GEP’s terms, in 32 bits
Device-scope barrier	block sum 2× slower	workgroup-scope barrier
LLVM 24: strength reduction, misplaced merge	crash, invalid SPIR-V	codegen at -O0; IR already optimized
Pointer phis from Mojo’s optimizer	compile error	phis of element indices
No print hook for debug_assert	compiler crash	print hook that drops output
Integer store of float bits	invalid SPIR-V	bitcast to the element type
No math hooks	logits off by 2.1e-4	exp, sqrt, … as GLSL.std.450

> Six were gaps in our new target; one is two LLVM 24 bugs.

Native target vs retargeter, same driver

Kernel	Native target	mojo2vk	Check
Vector add	1.6–1.7	1.7–2.2	pass
vec4 add	11.1	10.0	pass
Warp sum	2.31	2.30	pass
Block sum	**6.3**	13.7	pass
Naive SGEMM	670	670	pass
Shared-tile SGEMM	230	234	pass
fp16-tile SGEMM	**183**	216	pass
Register-tile SGEMM	34.4	34.9	pass

Faster where barriers dominate: the native barrier orders shared memory only.

Mojo and MAX on Adreno · 17 · Mesa Turnip · ms, median of 20 runs · SGEMMs 1024^3 · vector add varies run to run

LLAMA 3.2 1B, EVERY KERNEL FROM MOJO'S OWN COMPILER

283.6 ms
per token natively, against 285.0 through the retargeter

48/48
tokens identical to PyTorch

6.2e-5
largest logit error, the same as before

> Same weights, same 398-line runtime, same Turnip driver. Only the compiler changed.

Mojo and MAX on Adreno · 18 · same session, same phone · 46-token prompt: 11.35 s native, 11.37 s retargeted

Act 5 · Is MAX itself portable now?

MAX layer	On the Adreno today
Mojo compiler and stdlib	**Yes** · the adreno710 target
max.gpu: ids, barriers, warp ops	**Yes** · through the plugins
Kernel library: matmul, softmax, conv…	**Yes** · all 25 test graphs run
Graph compiler	**Yes** · on the Mac, for the phone
Runtime: devices, memory, launches	**Remote** · our plugin, from the Mac over adb
Python API, pipelines	**No** · needs Python on Android, or the C API

> MAX itself launches the kernels now, from the Mac. Running on the phone alone is next.

Compile test: MAX graphs for vulkan:adreno710

25/25

graphs give valid SPIR-V for every kernel: 211 kernels, checked by spirv-val

Graphs	Status
Eight elementwise ops, cast, transpose, slice, concat, gather	**Valid** · 13 graphs
sum, mean, argmax, softmax, layer_norm	**Valid** · 5 graphs
Three matmuls, conv2d, max and avg pool	**Valid** · 6 graphs
ResNet-18, the full graph	**Valid** · 74 kernels

From 16 to 25: big arguments read from memory, pointers that pick a buffer at run time, structs accessed by parts, alignment checks, memset loops, inlined asserts, wide vectors split, and structured control flow before LLVM’s own structurizer, plus one LLVM patch.

First MAX kernels on the phone

43/43

kernels from 13 MAX graphs match NumPy on the Adreno

Kernels	Result
add, mul, relu, exp, tanh, gelu, sigmoid, silu · 4 variants each	**Pass** · error ≤ 3.8e-6
f32→f16 cast, transpose	**Pass** · 1 f16 ulp, exact
Naive matmul 256^2 in f32 and f16, GEMV 1×2048^2	**Pass** · f32 error 4.5e-5

Each SPIR-V module now carries a launch description: where every pointer and number of the kernel's arguments goes. A test script plays MAX's runtime, and one generic launcher runs any kernel.

> It caught a real bug: MAX launches matmul in 16×16 blocks, but Vulkan had a fixed 64×1×1. The size is now set per launch.

MAX runs ResNet-18 on the phone

MAX on the Mac
graph compiler, host code, HAL device layer

Driver plugin
MAX's C interface; launches laid out from each kernel's description

Phone over adb
a small Vulkan executor on the Adreno 710

25/25
MAX graphs run on the phone through MAX's own runtime and match

5.7e-5
ResNet-18's largest difference from MAX on the CPU

8.3 s
per image, all GPU time now: MAX's naive conv2d dominates

Matrix hardware: KHR cooperative matrices

```
acc = CoopMatrix[DType.float16, 64, 64, ACCUMULATOR].zero()
for k in range(0, K, 16):
    a_tile = CoopMatrix[DType.float16, 64, 16, MATRIX_A].load(a + row * K + k, K)
    b_tile = CoopMatrix[DType.float16, 16, 64, MATRIX_B].load(b + k * N + col, N)
    acc = a_tile.multiply_add(b_tile, acc)
acc.store(c + row * N + col, N)  # --target-accelerator adreno840
```

Mojo
each operation is a marker call; the matrix travels as a handle

Kernel ABI pass
buffer and index for each load and store; handles typed through the K loop

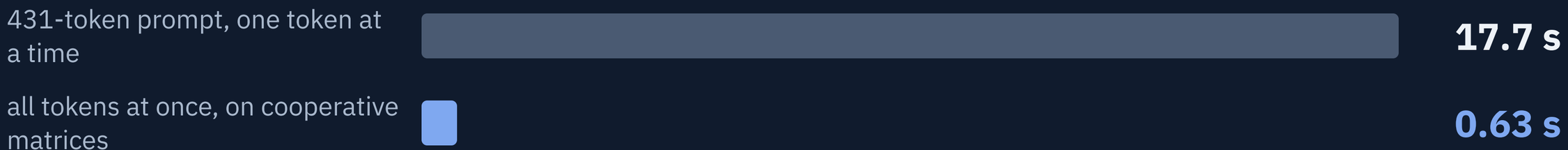
LLVM, patched
OpCooperativeMatrix*KHR in a shader, with the Vulkan memory model

2.1 TFLOP/s
fp16 matmul, 2048^3, on an Adreno 840 (Galaxy S26 Ultra, remote)

1.3 TFLOP/s
fp32, same size; 64×64 tiles, 3.6× the 64×16 ones

3/3
MAX’s matmul graphs pass on the phone, 8× its naive fp16 kernel

LLAMA 3.2 1B ON AN ADRENO 840: THE PROMPT ON THE MATRIX UNITS



28×

faster prompt processing, same phone and driver

48/48

tokens identical to PyTorch, logits within 6.0e-5

13 ms

per generated token with 4-bit weights; 43 in fp16, 285 on the Adreno 710

> Each projection is one matrix product for all 512 rows: fp32 cooperative matrices, the fp16 weights converted as they are staged in shared memory, 1.8 TFLOP/s; attention fused on the matrix units too, 46 ms.

Mojo and MAX on Adreno · 24 · llama/kernels/prefill · Galaxy S26 Ultra in Samsung’s Remote Test Lab · prompt: llama/prompts/long.txt

BONSAI 2 2B ON AN ADRENO 840: SOMEONE ELSE'S MOJO KERNELS, UNCHANGED

`kernels.mojo`
from mav3ri3k/modular's Android port, written for its OpenCL target

→

`--target-accelerator adreno840`
our native target: all 21 kernels to Vulkan SPIR-V, not one line changed

→

`bonsai_vk.c`
the port's forward pass in C: 721 dispatches a token, one command buffer

80 tok/s
decode, 12.4 ms a token; the port reports about 75 through OpenCL on an S26

6×
faster prompt: 412 tokens in 0.87 s instead of 5.2 s, on the matrix units

16/16
greedy tokens identical to the FP32 reference, token by token or batched; every logit checked

> Their kernels, unchanged, again for the prompt: one dispatch runs a decode kernel on every row. New: the ternary projections and attention scores on cooperative matrices, the DeltaNet recurrence as one scan.

Mojo and MAX on Adreno · 25 · bonsai/ · kernels: mav3ri3k/modular · decode: Galaxy S26 Ultra · prefill: Qualcomm's SM8850 reference phone

What VK_QCOM_cooperative_matrix_conversion would add

Weights straight into the matrix

One 32-bit word is 16 ternary codes, one fp16 K16 column: each lane builds its column in registers. No shared-memory staging, no barrier.

fp16 speed, fp32 accuracy

The 840 sums fp16 products only in fp16. Pull the accumulator out every 128 values and add it into fp32: 3.5 TFLOP/s fp16 instead of 1.8 fp32.

Attention without the round trip

Softmax between $Q \cdot K^T$ and $P \cdot V$ stays in registers, instead of the scores going through shared memory.

Integer products

Ternary weights are exact in int8: int8 products, the group scales applied in registers. Activations in int8 would cost accuracy.

Model	Today, KHR only (measured)	With conversion (estimate)
Llama 3.2 1B, 431-token prompt	0.63 s, products 0.56 s in fp32	**about 0.4 s**: fp16 products, fp32 sums
Bonsai 2 2B, 412-token prompt	0.87 s, products 0.80 s in fp32	**about 0.5 s**: no staging, fp16 products
Ternary Qwen3-8B, 408-token prompt	3.95 s, products 3.6 s in fp32	**about 2.4 s**: no staging, fp16 products

Mojo and MAX on Adreno · 26 · Vulkan 1.4.342 · not on the drivers we tested (512.842.19, .25) · runners load qcom/*.comp kernels when present

Porting MAX’s runtime: where it stands

1 **A Vulkan driver plugin**
Done, on the phone: MAX’s C interface on Vulkan, 6.5 µs a launch. The remote device shares its code.

2 **An Android build of the runtime**
Done: --config=android-arm64, from the Android OpenCL port, plus NDK r30 and stdlib fixes.

3 **Compile on the Mac, run on the phone**
Done for Mojo host code: programs carry their kernels as SPIR-V. Next: MAX graphs as .mef files.

4 **Kernel coverage, then tuning**
All 25 test graphs, ResNet-18 included, run correctly. Next: speed, and tuning per driver.

Mojo host code on the phone

```
model.mojo (DeviceContext launches) → MAX's HAL DeviceContext → libM_driver_vulkan.so → Vulkan
```

Model, its forward pass in Mojo	Mojo DeviceContext	C runner	Checked
Llama 3.2 1B, fp16	**24.7 tok/s**	23 tok/s	PyTorch 48/48
Llama 3.2 1B, 4-bit	**78.1 tok/s**	75 tok/s	PyTorch 48/48
Bonsai 2 2B: the port's own model.mojo	**74–76 tok/s**	80 tok/s	MLX 16/16
Qwen3-8B, ternary	**25.1 tok/s**	25 tok/s	MLX 16/16
Llama 3.2 1B: 431-token prompt, batched	**0.65 s** · 4-bit 0.59 s	0.62 s	PyTorch 48/48

The weights never cross adb: the phone downloads each checkpoint from Hugging Face and converts it, byte for byte the Mac's export.

> A launch costs 6.5 µs, because the plugin batches 64 a submission: decode runs as fast as the hand-written C runners.

Long prompts: prefill and decode rates

Model	Prompt	Prefill, batched	Decode after it	Checked
Llama 3.2 1B, fp16	431 tokens	**620 tok/s** · 0.70 s	**24.5 tok/s**	PyTorch 80/80
Llama 3.2 1B, 4-bit	431 tokens	**670 tok/s** · 0.64 s	**75.3 tok/s**	PyTorch 80/80
Qwen3-8B, ternary	the same passage: 408	**103 tok/s** · 3.95 s	**21.1 tok/s**	token by token: 80/80
Qwen3-8B, ternary	1000 tokens	**106 tok/s** · 9.4 s	**17.0 tok/s**	token by token: 24/24

Qwen3’s prompt: attention on the matrix units

$Q \cdot K^T$ for 1000 tokens took 12.2 s with a score a thread; on cooperative matrices, 0.44 s. Of the passage’s 3.95 s, the ternary projections are now 3.6.

Decode slows as the cache grows

Every layer reads 8 KB of fp32 KV cache per position: 47.3 ms a token after the passage, 58.8 after 1000 tokens, 41 after a short prompt.

Mojo and MAX on Adreno · 29 · Mojo DeviceContext programs, warm runs · Galaxy S26 Ultra, Adreno 840, Android 16 · Llama’s attention: the per-row fallback

What we learned

LLVM IR is a workable bridge	A retargeter got Llama running on a GPU Mojo never targeted, exactly.
The losses were lowerings	One warp-sum lowering cost 4.9× on Llama; SPIR-V itself cost little.
Drivers decide	Two Vulkan drivers disagree on speed, and twice on correctness. Check every output.
A native target removes fragility	Same speed, no stand-in target, no text patching. SPIR-V's limits remain.
MAX's runtime was the long pole	A new GPU target took about 2,600 lines, the driver plugin 1,500 more; Mojo host code now runs on the phone.

What comes next

MAX graphs on the phone

Mojo DeviceContext code runs there now. Next, MAX’s graph executor for Android, then compiled graphs from the Mac.

Faster models

Prompts on the matrix units, now 28× (Llama), 6× (Bonsai) and 4.4× (Qwen3-8B) faster; next, fp16 products with fp32 sums.

Upstream

Send LLVM the structurizer fix, report its strength-reduction crash. Offer Modular the target.

> The question is no longer whether Mojo can reach a phone GPU. It is how much of MAX should follow it there.